Practical no. 12

Aim - Web Element Interaction and Synchronization Handling
Develop a script to automate interaction with various web elements (textbox, radio button, checkbox, dropdown, alert, and frames). Implement implicit and explicit waits for synchronization handling.

Tool: Selenium WebDriver (Java)

Theory – WebElement Synchronization in Selenium
WebElement synchronization is the process of making Selenium wait until a web element is ready before performing an operation such as clicking, entering text, or retrieving a value. Web pages are often dynamic, meaning elements may take some time to load, become visible, or become clickable. If Selenium tries to interact with an element before it is ready, it may result in exceptions such as NoSuchElementException, ElementNotInteractableException or ElementClickInterceptedException. Selenium provides different synchronization mechanisms to handle these situations.

Types of Synchronization

1. **Implicit Wait**

Implicit wait tells Selenium to wait for a specified amount of time when searching for an element.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

2. **Explicit Wait**

Explicit wait waits for a **specific condition** before continuing execution.
Example:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement element = wait.until(
   ExpectedConditions.visibilityOfElementLocated(By.id("username"))
);
```

Common conditions include:

- visibilityOfElementLocated()
- elementToBeClickable()
- presenceOfElementLocated()
- titleContains()
- alertIsPresent()

3. **Fluent Wait**

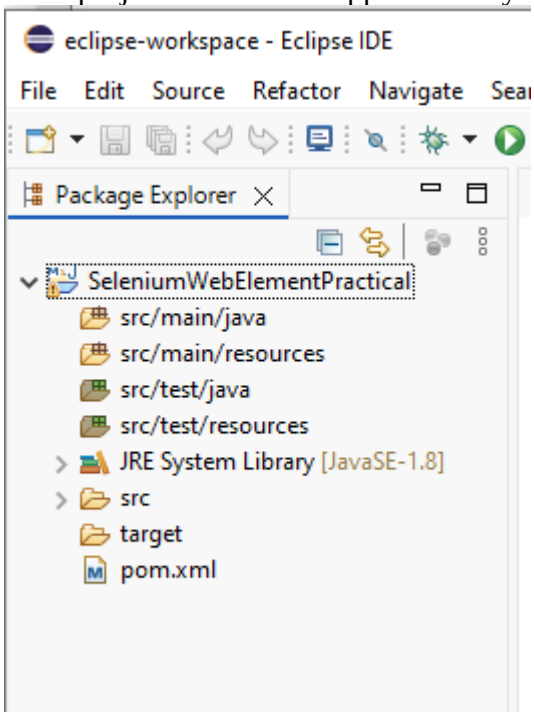
Fluent wait allows us to specify the timeout, polling interval, and exceptions that should be ignored while waiting.

Implicit Wait	**Explicit Wait**
Applies generally to element searches	Applies to a specific condition
Simple to configure	More flexible
Uses implicitlyWait()	Uses WebDriverWait
Mainly waits for element lookup	Can wait for visibility, clickability, alert, etc.
Global setting	Condition-specific

Implicit wait tells WebDriver to wait for a specified amount of time when searching for elements. **Explicit wait** waits until a specific expected condition is satisfied before continuing execution.

Step 1 - Check Java installation
Open **Command Prompt** and type:
java -version
javac -version
If both commands work, Java is installed correctly.

Step 2 - Create a Maven project in Eclipse
Open **Eclipse**.
Go to: **File → New → Maven Project**
Select: **Create a simple project (skip archetype selection)**
Click **Next**.
Enter:
Group Id: com.selenium
Artifact Id: SeleniumWebElementPractical
Packaging: jar
Click **Finish**.
Your project should look approximately like:



Step 3 - Add Selenium dependency
Open: pom.xml
You will see something similar to:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <groupId>com.selenium</groupId>
   <artifactId>SeleniumWebElementPractical</artifactId>
   <version>0.0.1-SNAPSHOT</version>
```

```
    <properties>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
      <maven.compiler.source>17</maven.compiler.source>
      <maven.compiler.target>17</maven.compiler.target>
    </properties>
    <dependencies>
      <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.35.0</version >
      </dependency>
    </dependencies>
</project>
```

Save this file.

Step 4 - Update Maven project

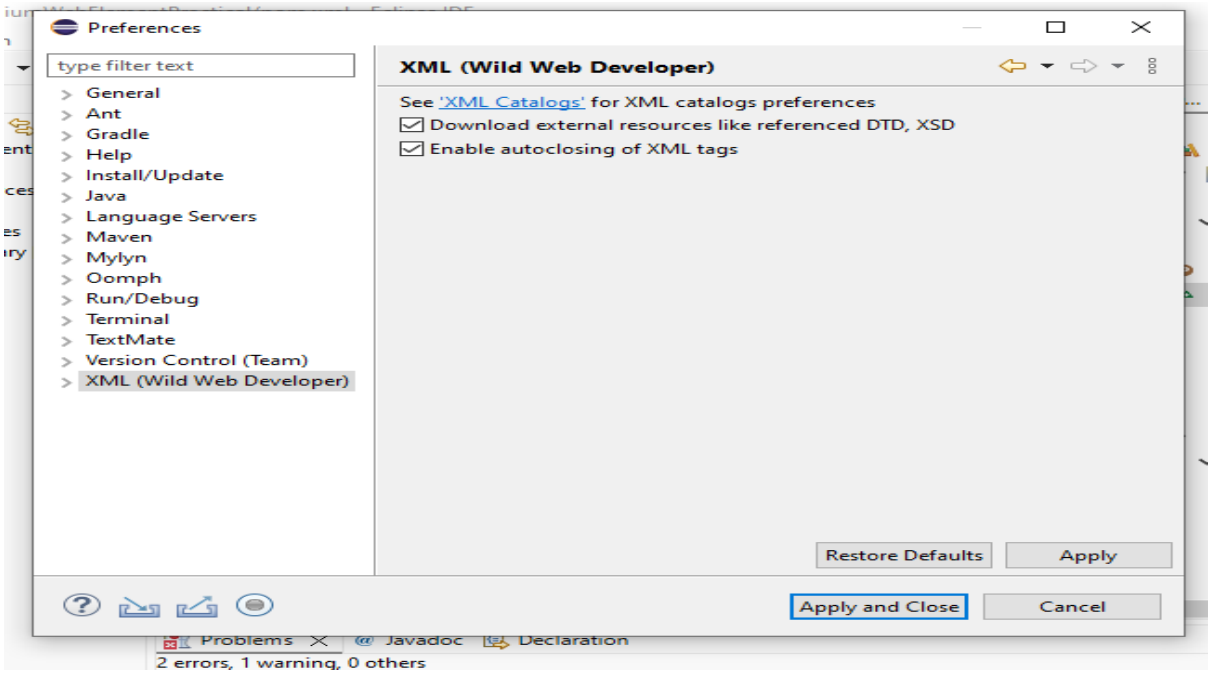
In Eclipse:
Right-click SeleniumWebElementPractical
→ **Maven**
→ **Update Project...**
→ select your project
→ **OK**
Wait for it to finish.

Check Maven Dependencies

Look at your project in Package Explorer.
Expand **Maven Dependencies**.
You should see Selenium JAR files.

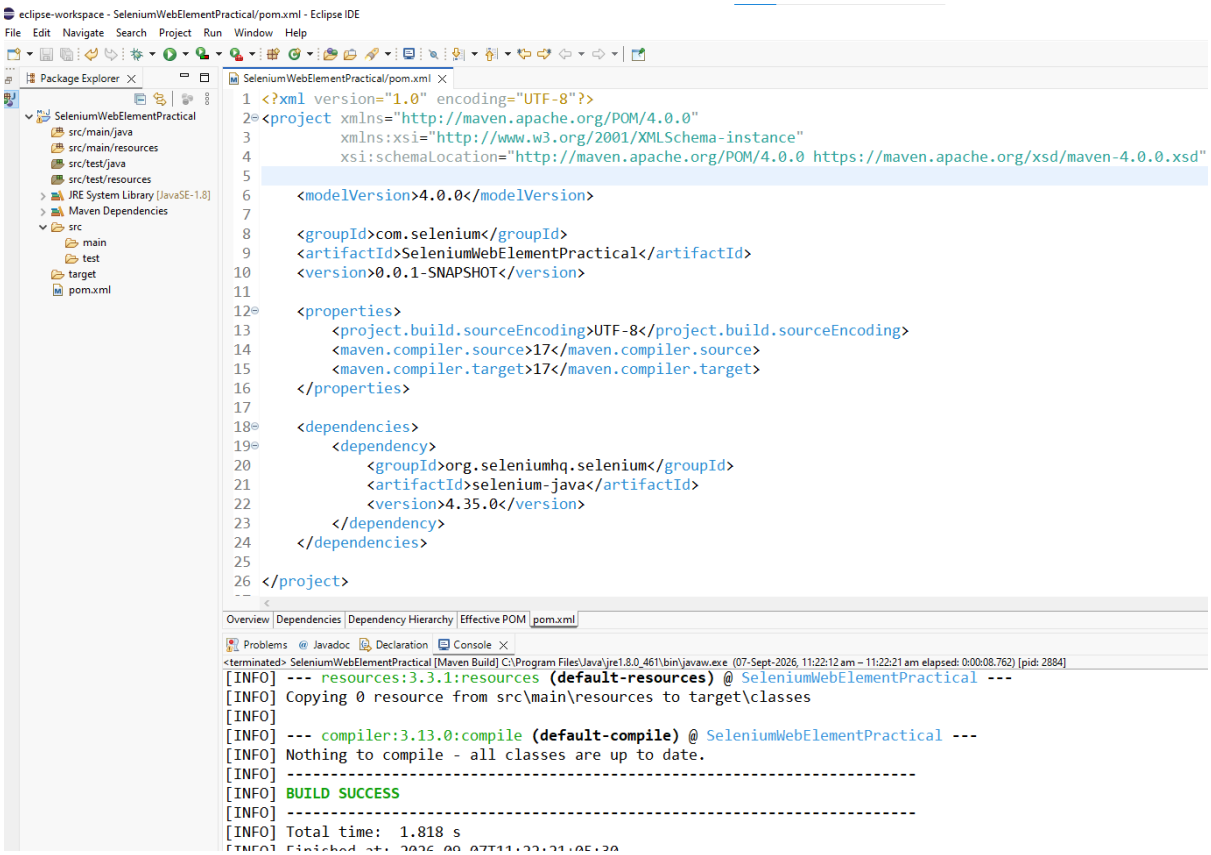
If the red error is STILL there

The message: Downloading external resources is disabled
means Eclipse's **XML Language Server** cannot access the Maven XML schema.
This is why Eclipse also shows: Cannot find the declaration of element 'project'
So don't keep changing the Selenium dependency. The problem is the XML validator.
Try this: **Window → Preferences**
Look for **XML(wild web developer)** → If you see an option related to **downloading external resources**, enable it.(tick) then apply then apply and close



Then: **Project → Clean... → Clean**
Finally: **left-click project → Maven → Update Project...**

Step 5 - Very important test
Even if the red XML error remains, check whether Maven actually works.
Right-click your project:
Run As → Maven build...
In **Goals**, enter: clean compile
Click **Run**.



Step 6 - Complete Selenium program
Right click on : src/test/resources
New-file- provide name- selenium-demo.html
Inside it create: selenium-demo.html
Put this HTML into the file:

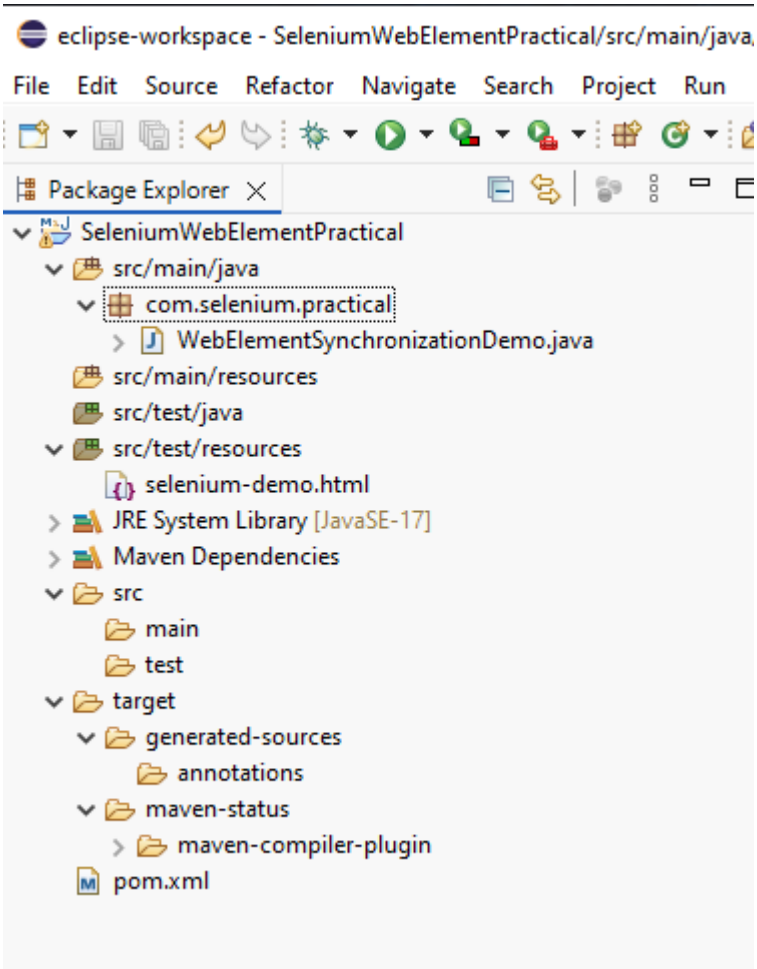
```
<!DOCTYPE html>
<html>
<head>
   <title>Selenium Web Element Practical</title>
   <style>
      body {
         font-family: Arial;
         margin: 40px;
      }
      .box {
         border: 1px solid #999;
         padding: 20px;
         margin-bottom: 20px;
      }
      iframe {
         width: 500px;
         height: 200px;
      }
   </style>
   <script>
      function showAlert() {
         alert("This is Selenium Alert");
      }
   </script>
</head>
<body>
<h1>Selenium Web Element Practical</h1>
<div class="box">
   <h2>Textbox</h2>
   <label>Name:</label>
   <input type="text" id="name" placeholder="Enter your name">
</div>
<div class="box">
   <h2>Radio Button</h2>
   <input type="radio" id="male" name="gender" value="Male">
   <label for="male">Male</label>
   <input type="radio" id="female" name="gender" value="Female">
   <label for="female">Female</label>
</div>
<div class="box">
   <h2>Checkbox</h2>
   <input type="checkbox" id="java">
   <label for="java">Java</label>
   <input type="checkbox" id="selenium">
   <label for="selenium">Selenium</label>
```

```
</div>
<div class="box">
  <h2>Dropdown</h2>
  <select id="course">
    <option value="">Select Course</option>
    <option value="java">Java</option>
    <option value="python">Python</option>
    <option value="selenium">Selenium</option>
  </select>
</div>
<div class="box">
  <h2>Alert</h2>
  <button id="alertButton" onclick="showAlert()">
    Click for Alert
  </button>
</div>
<div class="box">
  <h2>Frame</h2>
  <iframe id="myFrame"
      srcdoc="
      <html>
      <body>
      <h3 id='frameHeading'>Inside the Frame</h3>
      <input id='frameTextbox' type='text' placeholder='Frame Textbox'>
      </body>
      </html>">
  </iframe>
</div>
</body>
</html>
```

Step 7 - Create the Java class

Go to: src/main/java
Right-click: **New → Package**
Create: com.selenium.practical
Then right-click the package:
New → Class
Class name: WebElementSynchronizationDemo
Check: public static void main(String[] args)
Click **Finish**.
Your project now looks like:

```
SeleniumWebElementPractical
├── src/main/java
│   └── com.selenium.practical
│       └── WebElementSynchronizationDemo.java
├── Maven Dependencies
└── pom.xml
```



Step 8 - Write the Selenium Java code

Now open:

WebElementSynchronizationDemo.java

Use:

```
package com.selenium.practical;
import java.io.File;
import java.time.Duration;
import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.support.ui.WebDriverWait;
public class WebElementSynchronizationDemo {
   public static void main(String[] args) {
      // Launch Chrome
      WebDriver driver = new ChromeDriver();
      // Maximize browser
      driver.manage().window().maximize();
      // 1. IMPLICIT WAIT
      driver.manage().timeouts()
```

```
        .implicitlyWait(Duration.ofSeconds(10));
    // Open local HTML file
    File file = new File(
            "src/test/resources/selenium-demo.html"
    );
    driver.get(file.getAbsolutePath());
    // 2. EXPLICIT WAIT
    WebDriverWait wait =
            new WebDriverWait(driver, Duration.ofSeconds(10));
    // 3. TEXTBOX
    WebElement nameTextbox =
            wait.until(ExpectedConditions
                    .visibilityOfElementLocated(By.id("name")));
    nameTextbox.sendKeys("Rahul");
    System.out.println(
            "Textbox value entered successfully."
    );
    // 4. RADIO BUTTON
    WebElement maleRadio =
            wait.until(ExpectedConditions
                    .elementToBeClickable(By.id("male")));
    if (!maleRadio.isSelected()) {
       maleRadio.click();
    }
    System.out.println(
            "Radio button selected successfully."
    );
    // 5. CHECKBOX
    WebElement javaCheckbox =
            wait.until(ExpectedConditions
                    .elementToBeClickable(By.id("java")));
    if (!javaCheckbox.isSelected()) {
       javaCheckbox.click();
    }
    System.out.println(
            "Checkbox selected successfully."
    );
    // 6. DROPDOWN
    WebElement courseDropdown =
            wait.until(ExpectedConditions
                    .visibilityOfElementLocated(
                            By.id("course")));
    Select select =
            new Select(courseDropdown);
    select.selectByVisibleText("Selenium");

    System.out.println(
            "Dropdown option selected successfully."
    );
    // 7. ALERT
```

```
        WebElement alertButton =
                wait.until(ExpectedConditions
                        .elementToBeClickable(
                                By.id("alertButton")));
        alertButton.click();
        Alert alert =
                wait.until(ExpectedConditions.alertIsPresent());
        System.out.println(
                "Alert message: " + alert.getText()
        );
        alert.accept();
        System.out.println(
                "Alert accepted successfully."
        );
        // 8. FRAME
        WebElement frame =
                wait.until(ExpectedConditions
                        .presenceOfElementLocated(
                                By.id("myFrame")));
        driver.switchTo().frame(frame);
        WebElement frameTextbox =
                wait.until(ExpectedConditions
                        .visibilityOfElementLocated(
                                By.id("frameTextbox")));
        frameTextbox.sendKeys("Text inside frame");
        System.out.println(
                "Textbox inside frame accessed successfully."
        );
        // Switch back to main webpage
        driver.switchTo().defaultContent();
        System.out.println(
                "Returned to main webpage."
        );
        // 9. WAIT AND CLOSE
        try {
           Thread.sleep(2000);
        } catch (InterruptedException e) {
           Thread.currentThread().interrupt();
        }
        driver.quit();
        System.out.println(
                "Browser closed successfully."
        );
    }
}
```

Step 9 - Now execute the practical

Before running, check that your project looks like:

```
SeleniumWebElementPractical
├── src/main/java
│   └── com.selenium.practical
│       └── WebElementSynchronizationDemo.java
├── src/test/resources
│   └── selenium-demo.html
├── Maven Dependencies
└── pom.xml
```

Now:
Right-click WebElementSynchronizationDemo.java
→ **Run As**
→ **Java Application**

What will happen?
Chrome will open.
Selenium will perform the following operations automatically:
Step 1 — Open webpage
The browser opens:
selenium-demo.html
Step 2 — Textbox
Selenium finds:
<input type="text" id="name">
and executes:
nameTextbox.sendKeys("Rahul");
The textbox becomes:Rahul

Step 3 — Radio button
Selenium finds:
<input type="radio" id="male">
and clicks it.
maleRadio.click();

Step 4 — Checkbox
Selenium finds:
<input type="checkbox" id="java">
and checks it.
javaCheckbox.click();

Step 5 — Dropdown
Selenium finds:
<select id="course">
Then creates a Selenium Select object:
Select select = new Select(courseDropdown);
Then:
select.selectByVisibleText("Selenium");
selects:
Selenium

Step 6 — Alert

Selenium clicks:
Click for Alert
The JavaScript alert appears:
This is Selenium Alert
Selenium waits for the alert:

```
Alert alert =
    wait.until(ExpectedConditions.alertIsPresent());
```

Then accepts it:

```
alert.accept();
```

Step 7 —Frame
Selenium locates the iframe:

```
<iframe id="myFrame">
```

Then switches into it:

```
driver.switchTo().frame(frame);
```

Now Selenium can access the elements **inside the frame**.
It enters:
Text inside frame
Then it returns to the main page:

```
driver.switchTo().defaultContent();
```

13. Understanding Implicit Wait
This line:

```
driver.manage().timeouts()
      .implicitlyWait(Duration.ofSeconds(10));
```

is an **implicit wait**.
It tells Selenium:
If an element isn't immediately available, keep looking for it for up to 10 seconds.
For example:

```
driver.findElement(By.id("name"));
```

Selenium doesn't necessarily fail immediately if the element isn't found.
It can wait up to the configured timeout.
Syntax

```
driver.manage().timeouts()
      .implicitlyWait(Duration.ofSeconds(10));
```

14. Understanding Explicit Wait
We create:

```
WebDriverWait wait =
    new WebDriverWait(driver, Duration.ofSeconds(10));
```

Then we can wait for a **specific condition**.
For example:

```
wait.until(
    ExpectedConditions.visibilityOfElementLocated(
        By.id("name")
    )
);
```

This means:
Wait until the element with ID name becomes visible.
Another example:

```
wait.until(
    ExpectedConditions.elementToBeClickable(
        By.id("male")
    )
);
```

This means:
Wait until the radio button is clickable.
And:

```
wait.until(
    ExpectedConditions.alertIsPresent()
);
```

means:
Wait until the alert appears.

Step 10 - Expected output
Your Eclipse console should show something similar to:

